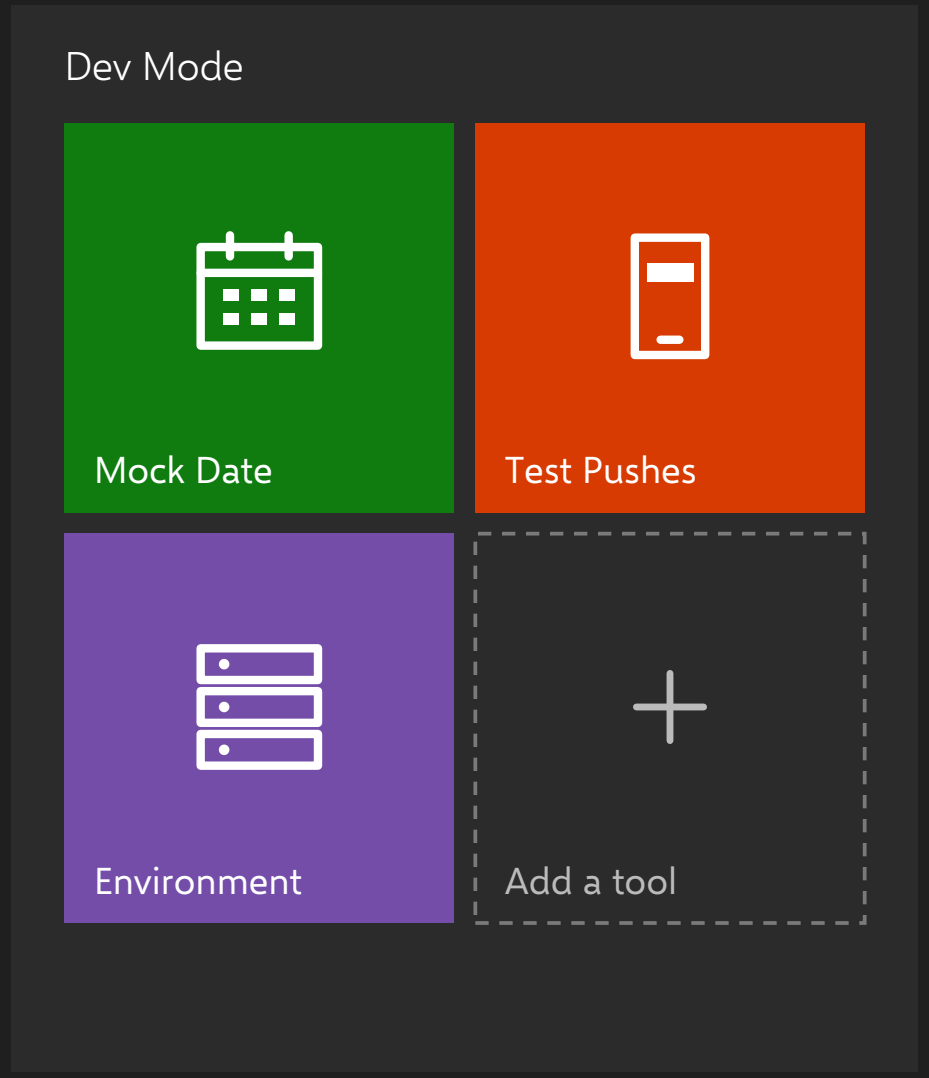
Conf42 Platform Engineering 2026

Dev Mode as a Plugin Framework

One place for all internal tools in a mobile app

Denis Galkin

November 5, 2026 · Online

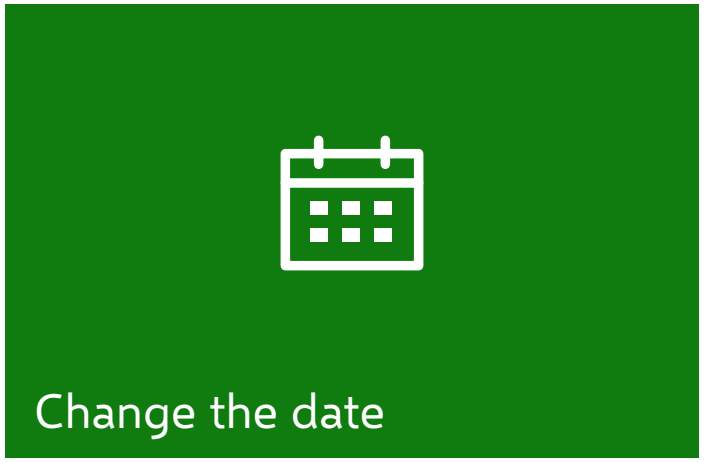


01 · Introduction

Internal tools every app has



Check what happens when a notification arrives.



See how screens behave on another day.



Point the app to another server.

Every mobile team has them. Very few teams design them on purpose.

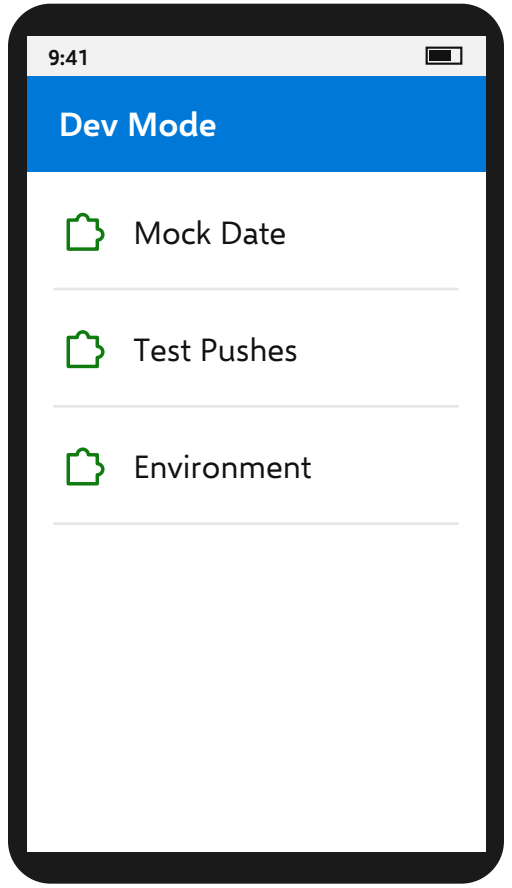
01 · Introduction

A platform can live inside an app

Platforms we usually talk about

- CI/CD
- Kubernetes
- Developer portals

and also



Dev Mode

a much smaller platform, inside a mobile app

01 · Introduction

Today



Part 1

The problem

What happens to internal tools when nobody designs them



Part 2

The solution

Dev Mode as a plugin framework, and how it works inside



Part 3

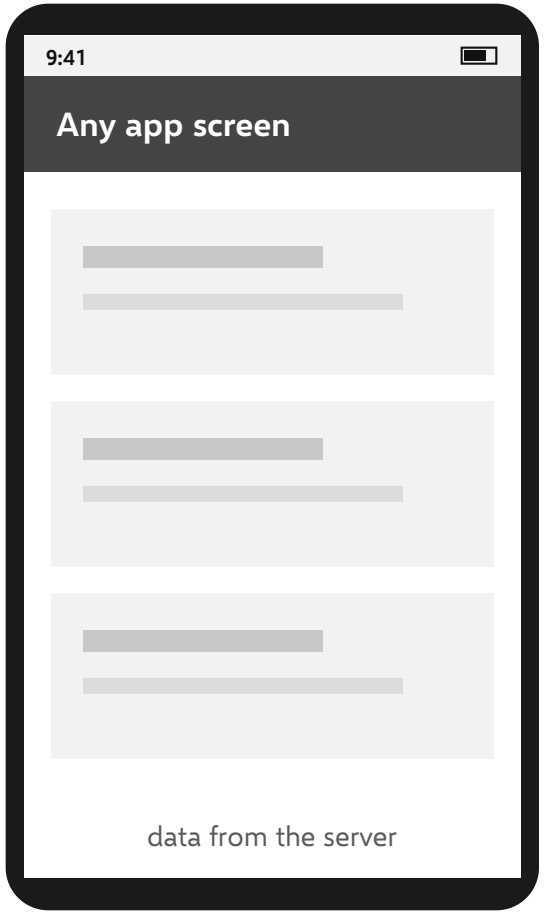
The lessons

Adoption without pressure, and what I would do differently

Examples from Android · ideas for any platform team

02 · The problem

To test a screen, you change its inputs



The date

A server response

A push notification

So developers build small tools for that.

02 · The problem

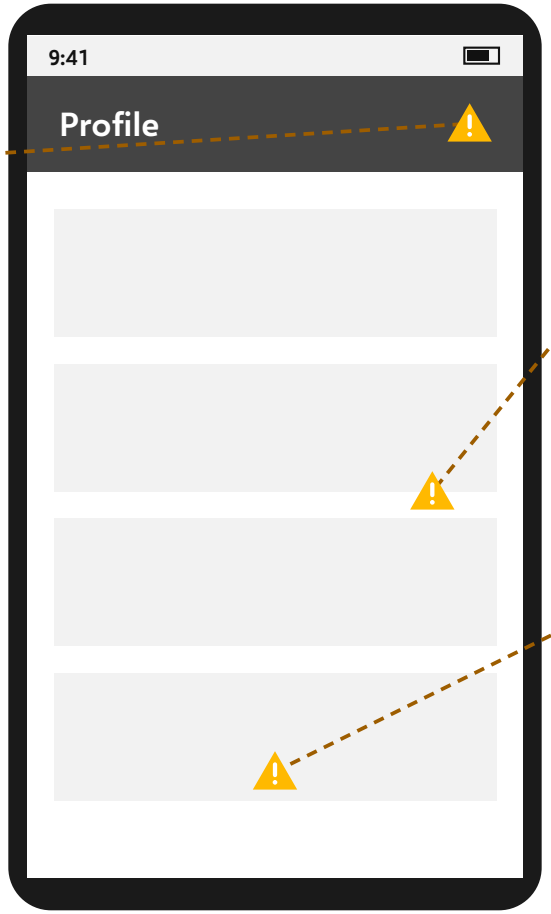
Tools grow without a plan

Hidden button
on the profile screen

Its own UI

Its own navigation

Its own way to open



Debug menu
in the settings

Secret gesture
somewhere else

Nobody planned it.
It just grows.

02 · The problem

Everyone pays for it

</>

Developers

Every new tool starts from zero.

Where do I put it? How do I open it? How do I go back?

QA engineers

They need these tools every day.

Where does each one live? Or ask a developer, who stops their work.

The company

Every app gets its own debug menu.

Different look, different logic, nothing shared.

Teams build the same thing many times, in different ways, and nobody owns the result.

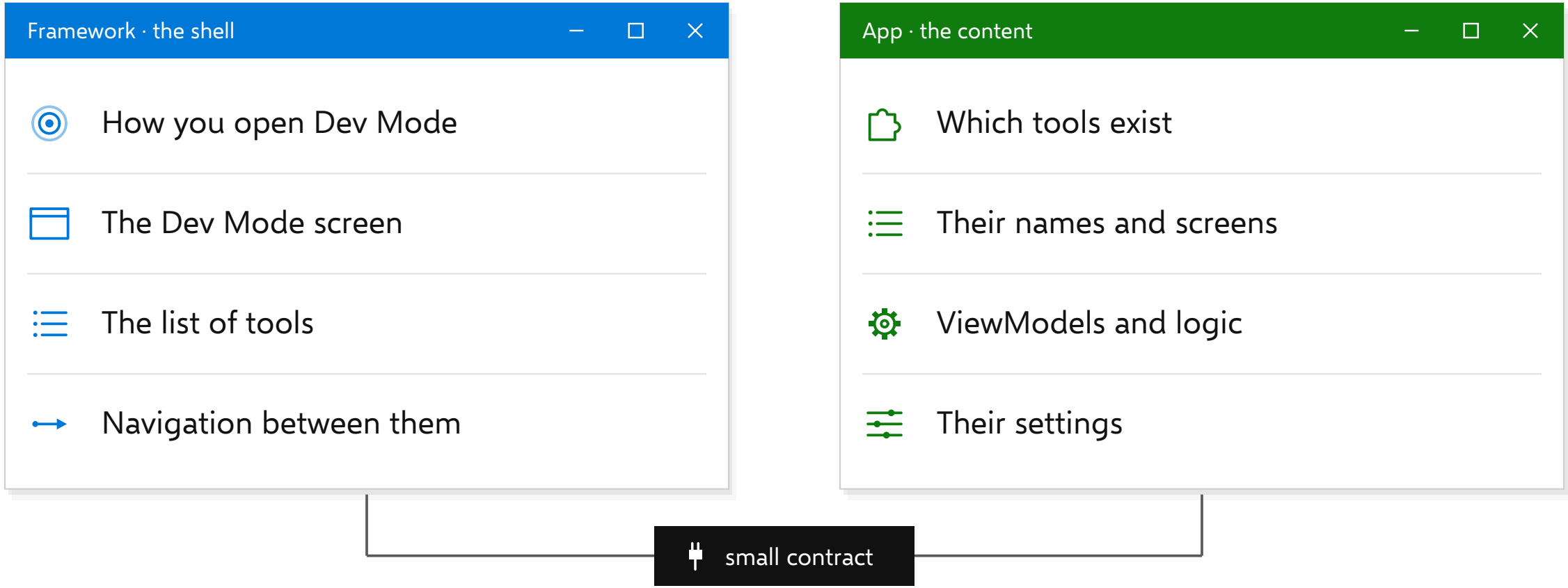
The classic platform engineering problem. Here it happens inside one app.

Our answer: Dev Mode as a plugin framework

Built early, before the tools had a chance to spread.

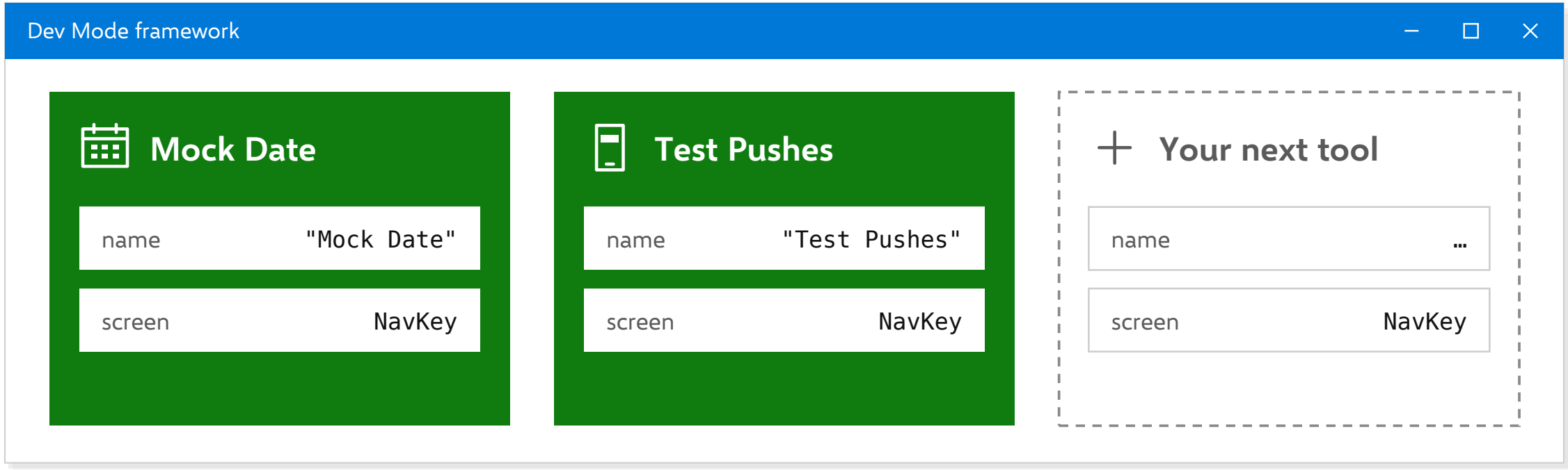
03 · The idea

One shell, many plugins



03 · The idea

A plugin is just a name and a screen



Like Backstage: a small core, and everything useful lives in plugins.

03 · The idea

The two parts change at different speeds

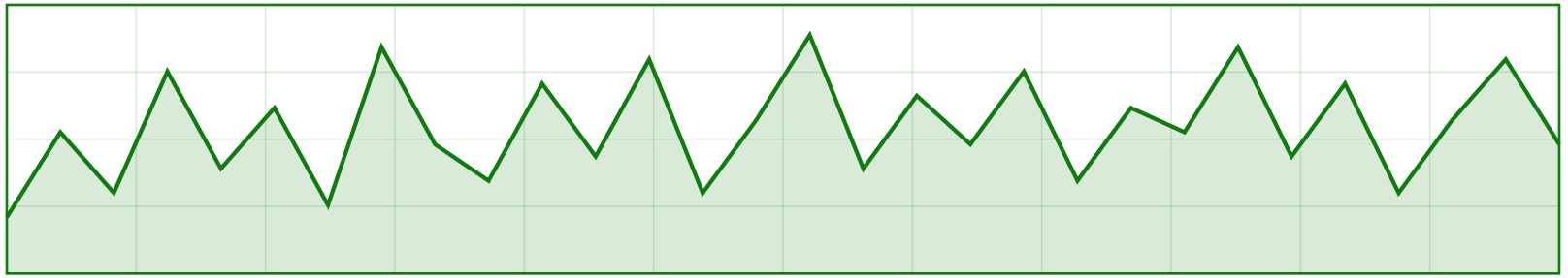
Shell · framework

almost never changes



Content · plugins

changes all the time



Mixed together

Every new tool touches shared code.

Separated

Shared code stays quiet. All changes happen in the plugins.

03 · The idea

The deal

Framework

"Give me your plugins, and I'll show them in one place, in one consistent way."

App

"Here are my plugins."

04 · Opening Dev Mode

Opening Dev Mode: one line in the app

MainActivity.kt · Android View

```
1 findViewById<View>(R.id.devModeButton)?.initDevModeListener()
```

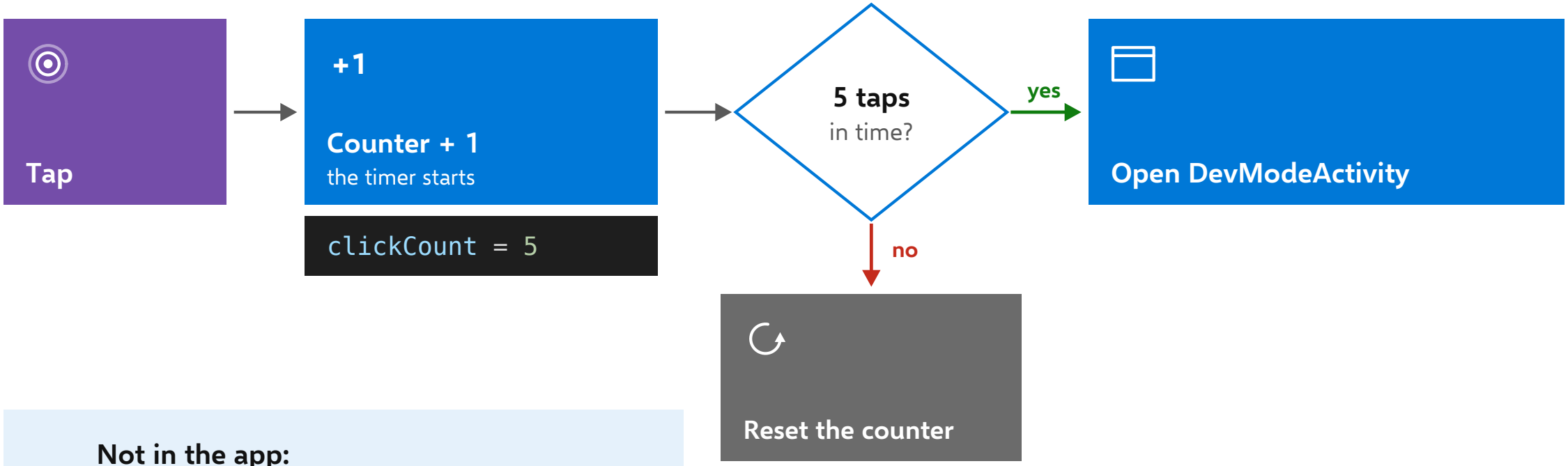
Jetpack Compose

```
1 Modifier.devModeClickable(...)
```

The app only says: “this element opens Dev Mode”. Nothing more.

04 · Opening Dev Mode

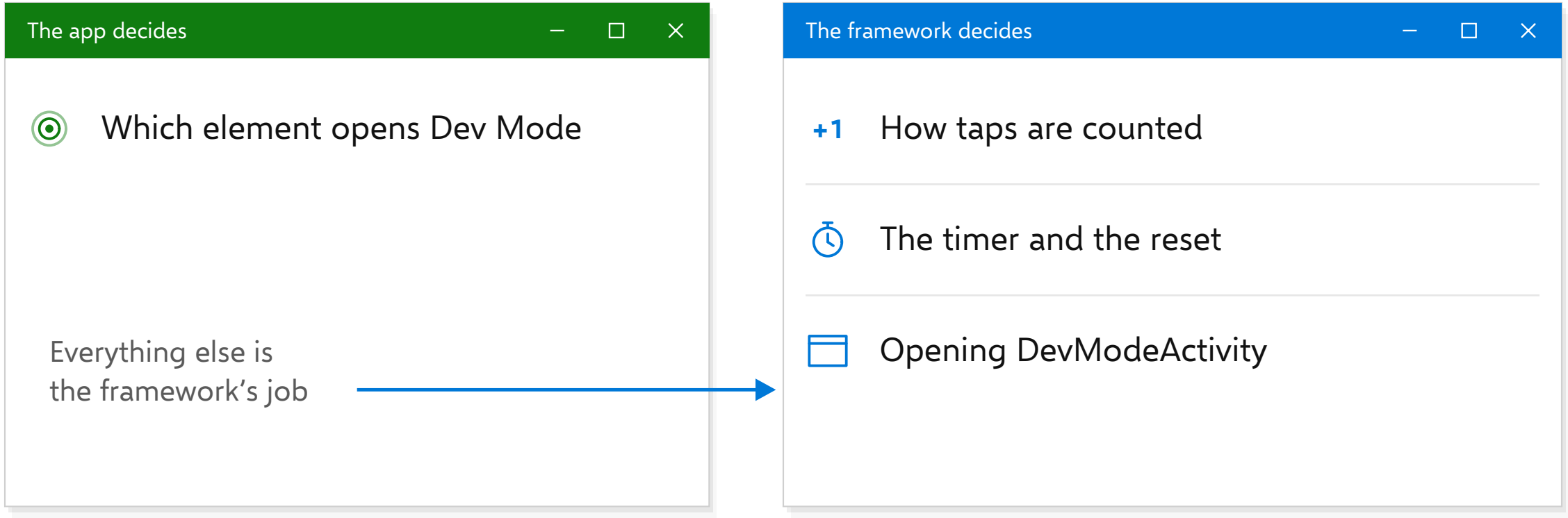
DevModeRunner: five taps in time



Not in the app:
the counter, the timer, the reset, or opening Dev Mode.

04 · Opening Dev Mode

The app makes only app decisions



A small detail that shows the main rule.

DevModeActivity doesn’t know which plugins exist.

The most important design decision in the framework.

05 · The contract

The contract

IDevModePluginEntriesProvider.kt

```kotlin
interface IDevModePluginEntriesProvider {
    fun entries(
        onBackPressed: () -> Unit
    ): EntryProviderScope<NavKey>.() -> Unit
}
```

Properties

Defined by
the framework

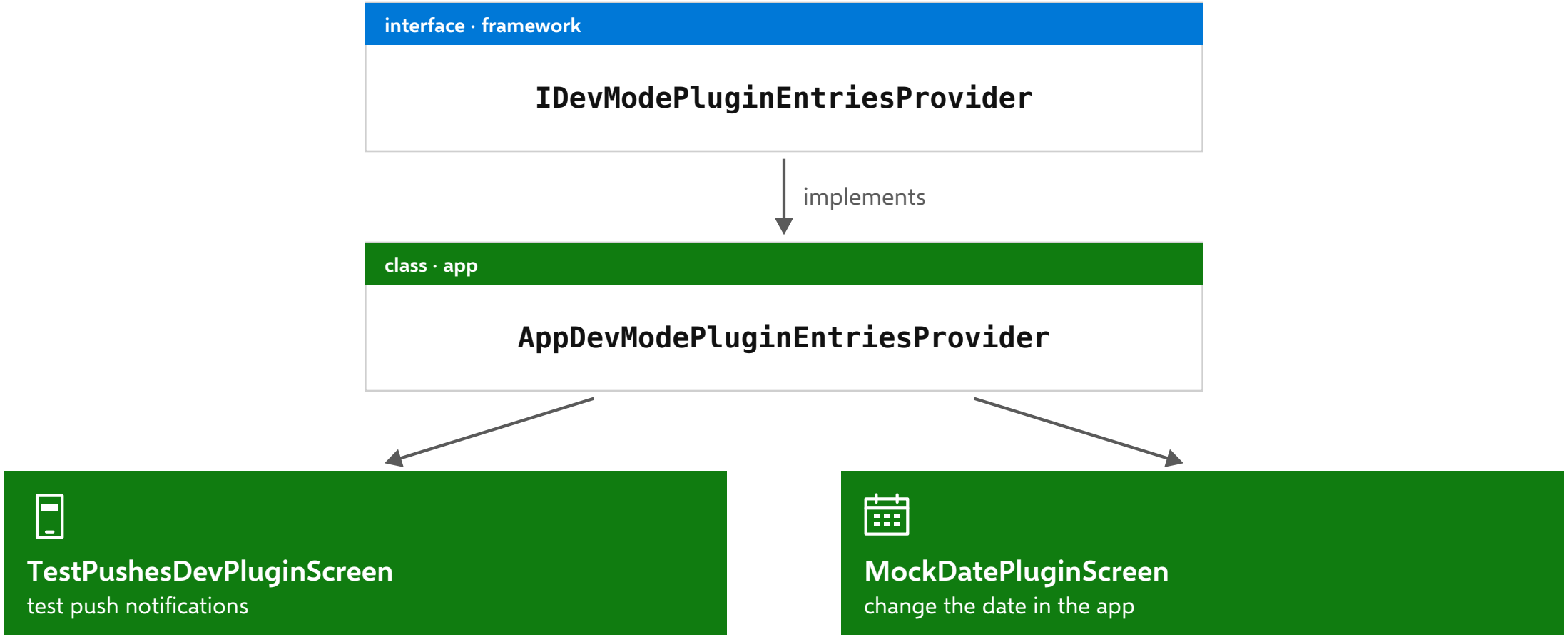
Implemented by
the app

Returns
Navigation 3 entries

> “If you want to add screens to Dev Mode, register them here.”

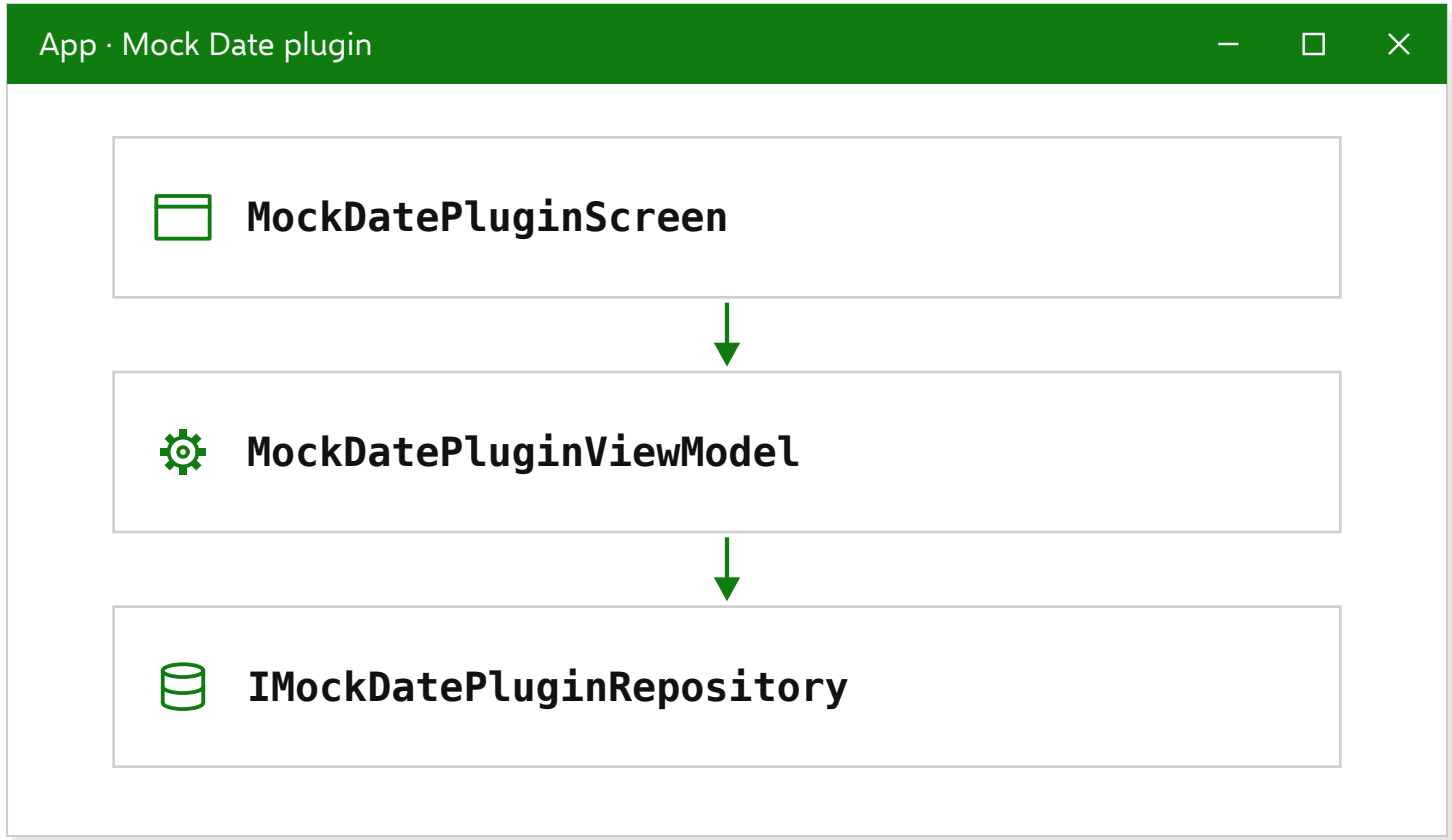
05 · The contract

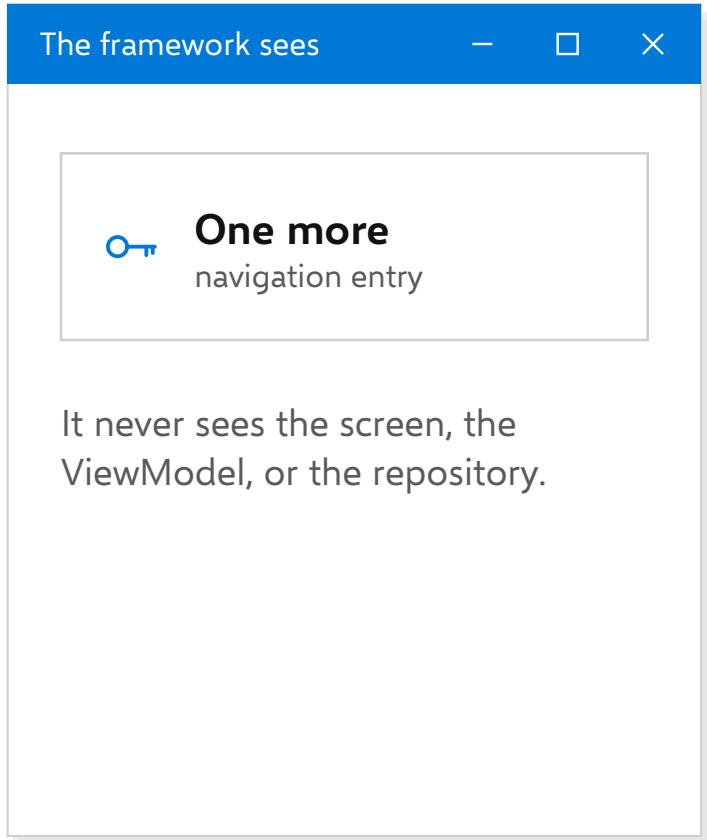
The app implements it



05 · The contract

Inside a plugin: Mock Date





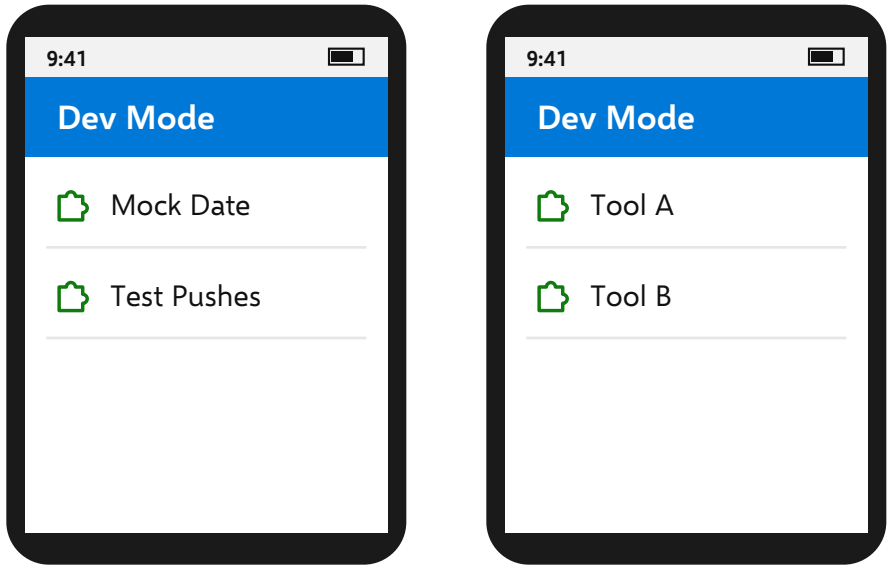
05 · The contract

A clear boundary, and reuse for free

File Explorer

- **devmode** framework
  - DevModeActivity.kt
  - DevModeRunner.kt
  - DevModeRouter.kt
  - DevPluginsScreen.kt
  - IDevModePluginEntriesProvider.kt
- **devmode.plugins** app
  - AppDevModePluginEntriesProvider.kt
  - MockDatePluginScreen.kt
  - TestPushesDevPluginScreen.kt

Another app? Its own provider, its own screens.

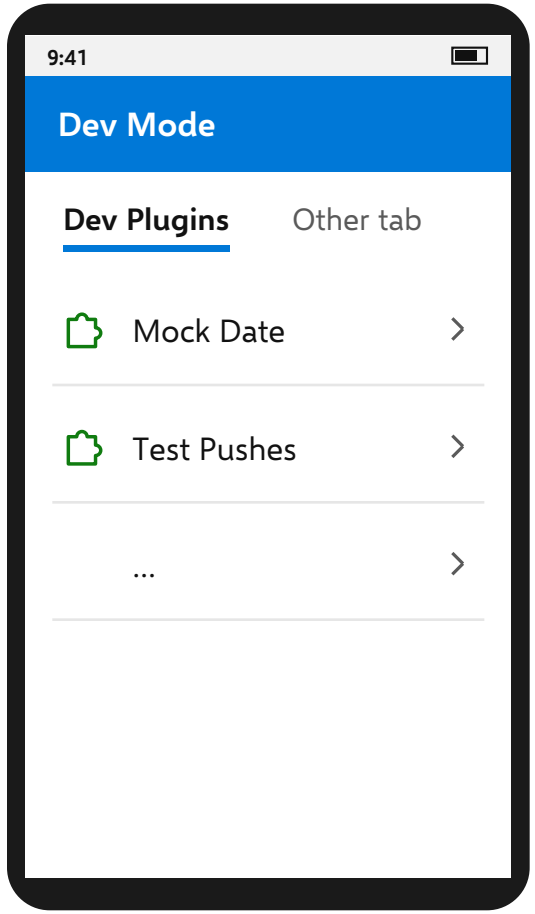


App 1 App 2

In platform terms, this interface is our API.

06 · The list and navigation

The Dev Plugins list



IDevPluginsProvider
the app provides the list

↓

List<DevPluginDTO>

↓

DevPluginsViewModel

↓

DevPluginsScreen
LazyColumn

DevPluginDTO · Properties

index	`0`
title	`"Mock Date"`
screen	`NavKey`

"Here is a menu item. This is its name. Tap it to open this screen."

06 · The list and navigation

From tap to navigation

Navigation flow (simplified)

```
// DevPluginsScreen: LazyColumn item
onClick = { viewModel.onPluginClick(it) }

// DevPluginsViewModel
router.routeToScreen(plugin.screen)

// DevModeRouter
fun routeToScreen(screen: NavKey) {
    router.navigateTo(screen)
}
```

DevModeRouter

- Takes a NavKey
- Navigates to it
- Doesn’t know which screens exist

06 · The list and navigation

No “when” in the framework

What the framework does NOT have

Not in our code

```
when (plugin) {
    MockDate    -> openMockDate()
    TestPushes -> openTestPushes()
    // ...one branch per tool
}
```

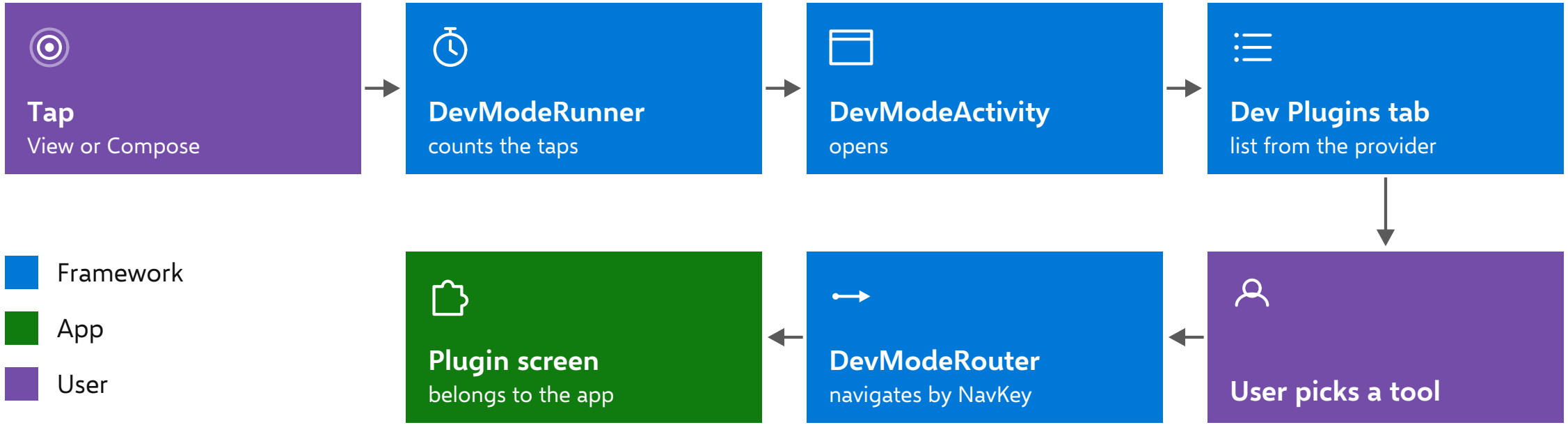
`when = Kotlin’s switch`

The framework never decides. It gets a key from the app and passes it to navigation.

06 · The list and navigation

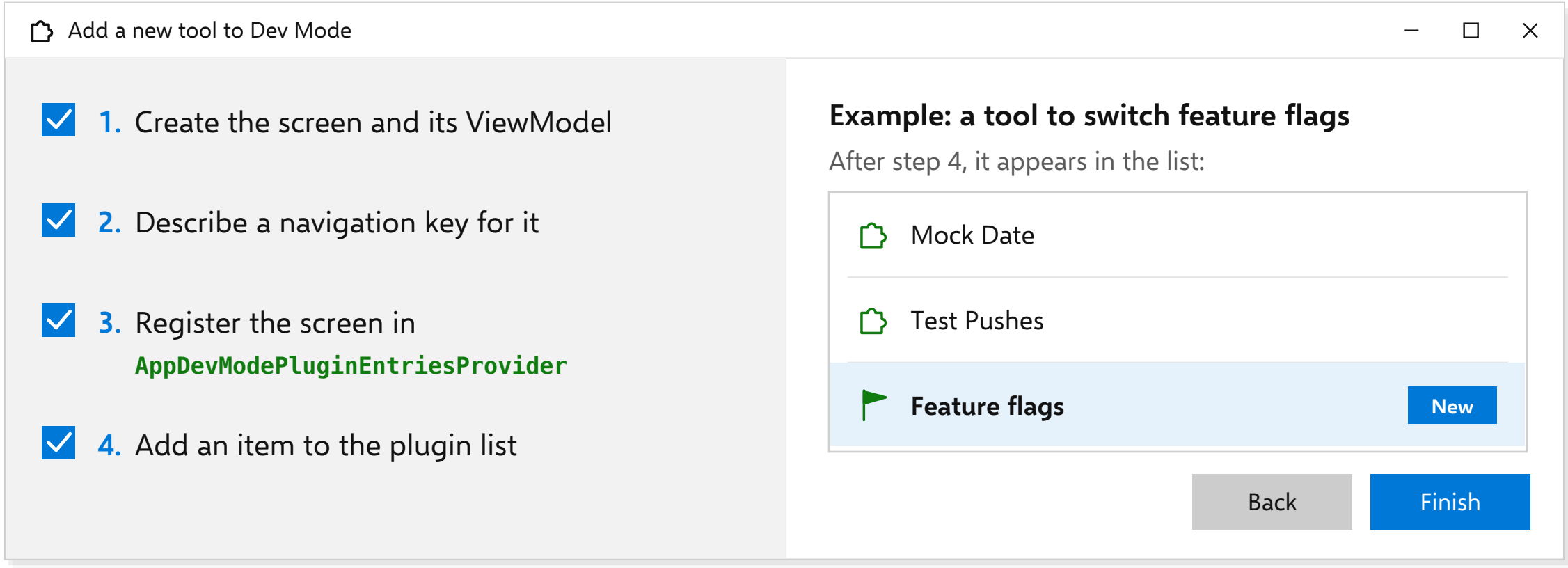
The full path

The framework builds the road. The app decides where it leads.



07 · Adding a tool

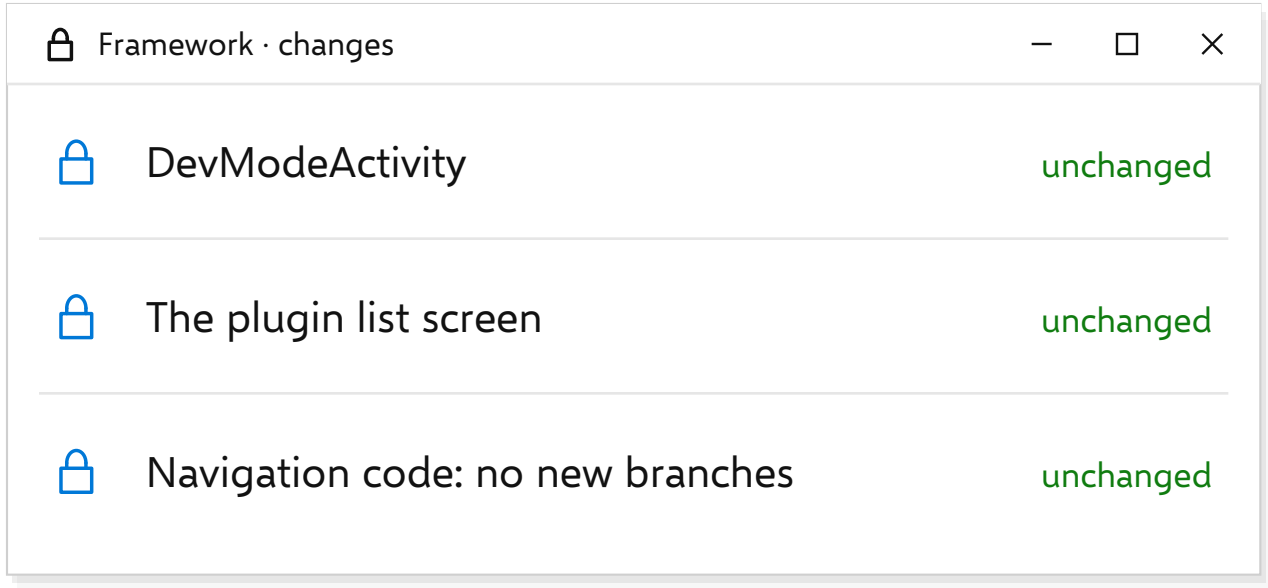
A new tool: four steps, all in the app



Same look, same navigation as every other tool.

07 · Adding a tool

What you don’t touch



0

changes in the framework

The cost of a new tool = the cost of the tool.
Two plugins or twenty: the framework stays the same.

08 · Who uses it

Not only for developers



Developers

One standard place for any debug tool.



QA engineers

Set up scenarios on their own: change the date, send a test push. No developer, no special build.

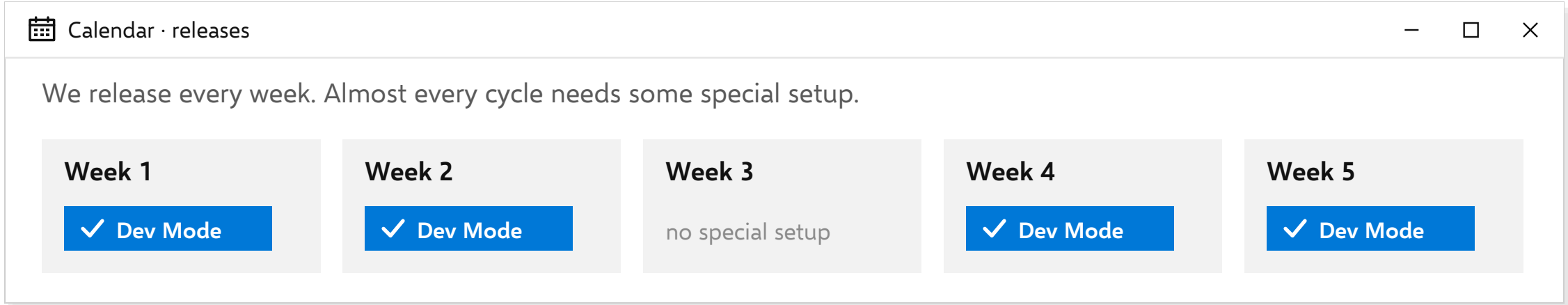


Managers

Check how the app behaves in different situations, without asking for help.

08 · Who uses it

Part of everyday work



Your users are not only engineers.

When other roles can work on their own, everyone saves time, including the engineers.

09 · Adoption

Adopted without a fight

- ✗ No migration
- ✗ No convincing
- ✗ No “you must use it” rule

Why? Three reasons:

1. **Timing**
2. **The golden path**
3. **Ownership**

09 · Adoption

Reason 1: timing

The cheapest moment to introduce a standard is before there's anything to migrate.



- ✓ Nothing to migrate
- ✓ Cost: 1 developer, 1 sprint

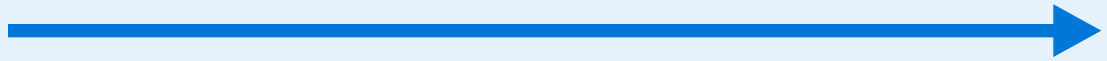
- ✗ Old debug menus to move
- ✗ Owners to convince
- ✗ Meetings and discussions

09 · Adoption

Reason 2: the golden path

Add a plugin
register the screen + add a menu item

minutes



Build your own debug screen
UI, navigation, a way to open it, from zero



Make the right way the easiest way.
You don’t force people onto it. They choose it themselves.

09 · Adoption

Reason 3: ownership follows the feature



That person knows best which data and states need to change.

- Core: small and stable
- Changes: in the plugins
- Almost no maintenance

10 · Lessons

If I started again: measure from day one

Metric 1

Time to prepare a test scenario

with Dev Mode and without it

?

not measured

Metric 2

How often QA needs a developer

for example, requests per week

?

not measured

We prevented the problem, so there is no “before”.

Start measuring early: even simple numbers help explain a platform’s value.

10 · Lessons

If I started again: an owner for the core

Today, different developers improve Dev Mode when they need something.

Risks as plugins grow

- ⚠ Plugin screens start to look different
- ⚠ Old tools stay long after anyone needs them

One owner, a few simple rules

- ✓ Who reviews changes to the core
- ✓ How a plugin screen should look
- ✓ When we remove old tools

ℹ **Today it works:** the core is small and stable.

11 · When it fits

When this approach fits

A great fit

- Many screens driven by server data
- More than one role tests the app
- Frequent releases
- Several apps: the same shell for free

The bigger the app, the more value you get.

Maybe too much

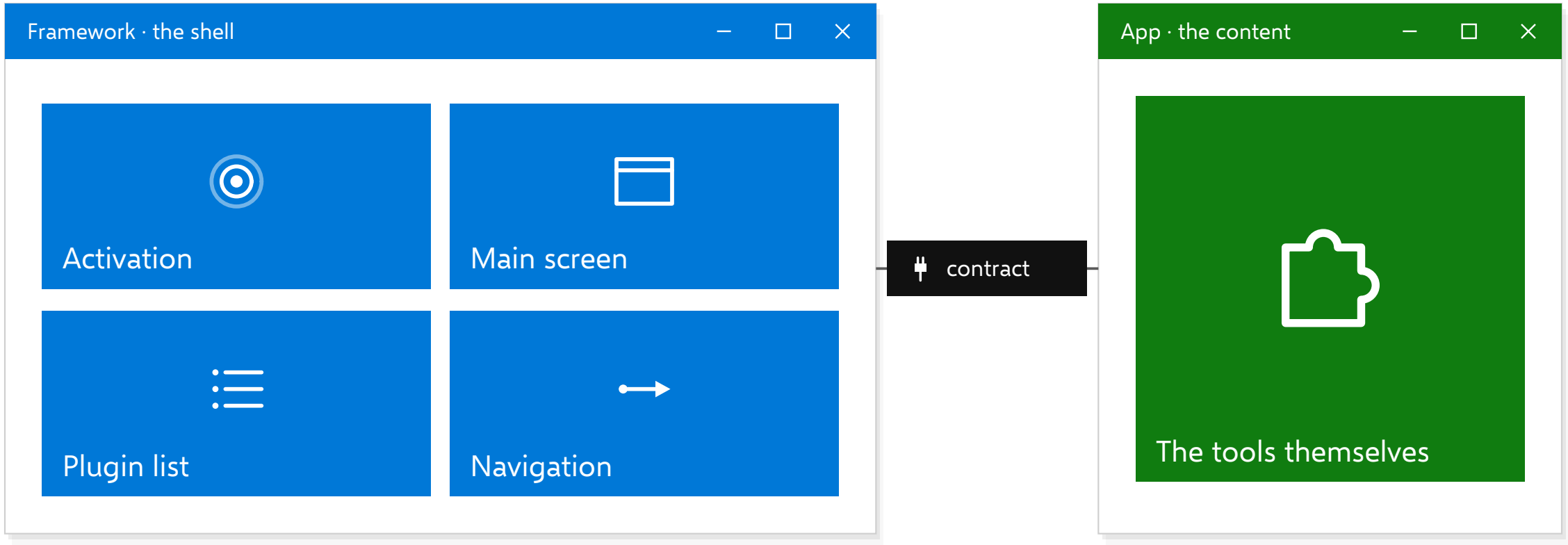
- A small app
- A short-lived prototype

Still keep one rule:
one entry point for all internal tools.

12 · Summary

Summary

Internal tools grow in every app. Without a system, everyone pays for it.



12 · Summary

Three lessons for any stack

1

Build the shell early

before there's anything to migrate.

2

Make the right way the easiest way

then you don't need rules to make people follow it.

3

Design for all your users

not only developers.

Thank you!

Questions, or your own story about internal tools?

Let’s connect on LinkedIn →

Conf42 Platform Engineering 2026 · November 5, 2026

Contact card



Denis Galkin

Android developer · 7+ years

RoboFinance

Kotlin · Compose · Hilt · Coroutines

Scan to open my LinkedIn

or search for Denis Galkin, RoboFinance